Home Networking Lab: Build Guide

Liam Rose, April 2026

Repository: https://github.com/LiamSeanRose/home-networking-lab

This is the full record of how I built the lab, all four phases in one place. I built it over the summer because I wanted to learn things that weren't being taught in class, like infrastructure as code, EVPN, automation and monitoring, and do it as cheaply as I could on one PC. I wrote this guide so I could rebuild everything from a blank PC, and so someone else could too. Every config file mentioned here is in the repo.

Contents

1. Overview
2. Hardware and addressing
3. Phase 1: Proxmox and Terraform
4. Phase 2: Containerlab and VXLAN-EVPN
5. Phase 3: NAPALM and pyATS
6. Phase 4: Zeek, ELK and Tailscale
7. Operations and recovery
8. Troubleshooting
9. Running it on other hardware
10. Possible extensions

1. Overview

Each phase runs on top of the one before it.

Phase	What gets built	Main skills
1	Proxmox hypervisor, Ubuntu cloud-init template, Terraform pipeline that creates the lab VM	virtualization, infrastructure as code, cloud-init, API tokens
2	Spine-leaf fabric of Arista cEOS routers in Containerlab: eBGP underlay, EVPN overlay, VXLAN	BGP, route reflection, EVPN route types 2 and 3, VXLAN
3	Python scripts that read BGP state and push config, plus a pytest/pyATS test suite	NAPALM, pyATS/Genie, pytest, parallel connections, config diffs
4	Zeek sensor, Filebeat, Elasticsearch, Kibana dashboard, Tailscale remote access	network monitoring, log pipelines, zero-trust access

The flow from code to running network:

```
Windows workstation          Proxmox host (bare metal)    lab-vm-1 (Ubuntu 24.04)
─────────────────────        ─────────────────────────    ──────────────────────

terraform apply —API token—> clone template 9000 —boots—> Docker
                             cloud-init: user, SSH key    ├— Containerlab: 3 cEOS + 2 hosts
                                                          ├— Elasticsearch + Kibana (Compose)
                                                          ├— Zeek + Filebeat (host services)
                                                          └— Tailscale
```

2. Hardware and addressing

Item	Value
Host PC	32 GB RAM, NVIDIA RTX 3070, dedicated NVMe
Hypervisor	Proxmox VE 9.1.7 (Debian 13 Trixie)
Proxmox hostname / IP	`pve.home.lab` / `192.168.1.4`
Proxmox web UI	`https://192.168.1.4:8006`
Lab VM	ID 100, name `lab-vm-1`, user `labadmin`
Lab VM IP	`192.168.1.80` (DHCP, reserved on the home router)
Workstation	Windows laptop (Surface Pro 8), runs Terraform

How the lab VM grew:

Point in project	RAM	vCPU	Disk
Phase 1 baseline	2 GB	2	20 GB
Phase 2	8 GB	4	40 GB
Phase 4	12 GB	6	40 GB

Each resize was just a variable change and `terraform apply`. I never had to recreate the VM.

Fabric addressing:

Item	Range
Management	172.20.20.0/24 on bridge `clab-mgmt`
Loopbacks	10.0.0.1 (spine1), 10.0.0.11 (leaf1), 10.0.0.12 (leaf2)
Spine-leaf links	10.1.1.0/31 (spine1-leaf1), 10.1.1.2/31 (spine1-leaf2)
Tenant A	VLAN 100, VNI 10100, hosts 10.100.100.11 and .12
Tenant B	VLAN 200, VNI 10200 (added by script in Phase 3)
Underlay ASNs	65000 spine1, 65001 leaf1, 65002 leaf2
Overlay ASN	65100 on all three (via `local-as`)

3. Phase 1: Proxmox and Terraform

Goal: create and destroy the lab VM from code. I wanted to be able to wreck the lab and get it back in under a minute, so I never had to be careful with it.

3.1 Install Proxmox VE

1. Write the Proxmox VE ISO to a USB stick and boot from it.
2. Pick the terminal UI installer from the boot menu. The graphical installer crashed on this machine, which I traced to the RTX 3070.
3. Choose the target disk, then set hostname `pve.home.lab`, static IP `192.168.1.4` with the home router as gateway and DNS, and a strong root password.
4. After the reboot, log in at `https://192.168.1.4:8006` with realm Linux PAM and user `root`.

On the physical console, log in as `root` with no `@pam` suffix (the suffix is only for the web UI). The console shows nothing while you type the password, which is normal.

3.2 Terraform user and API token

In the web UI:

1. Datacenter → Permissions → Users → Add. User `terraform`, realm "Proxmox VE authentication server" (pve, not pam), set a password, enabled.
2. Datacenter → Permissions → Add → User Permission. Path `/`, user `terraform@pve`, role `Administrator`. That's broader than it needs to be, but it's a lab.
3. Datacenter → Permissions → API Tokens → Add. User `terraform@pve`, token ID `terraform-token`, uncheck Privilege Separation so the token gets the user's rights.

Copy the token right away. It looks like `terraform@pve!terraform-token=<secret>` and Proxmox only shows it once. It is effectively root on the host, so it never goes in Git. If it leaks, delete it and make a new one.

3.3 Terraform and SSH key on the workstation

PowerShell:

```
winget install HashiCorp.Terraform
# reopen PowerShell
terraform -version        # needs 1.5.0 or newer

dir $HOME\.ssh            # check for an existing key
ssh-keygen -t rsa -b 4096 -f $HOME\.ssh\id_rsa -N '""'
type $HOME\.ssh\id_rsa.pub
```

`-N '""'` is how PowerShell passes an empty passphrase. With no passphrase, Terraform can use the key without prompting. cloud-init puts the public key on every VM.

3.4 Ubuntu cloud-init template (VM 9000)

A template is a base VM that Proxmox clones. Cloud-init sets the hostname, user, SSH key and network on each clone's first boot, so a new VM is ready in about 30 seconds instead of going through an installer.

Run on the Proxmox host (web UI → node → Shell):

```
mkdir -p /var/lib/vz/template/iso
cd /var/lib/vz/template/iso

# Ubuntu 24.04 cloud image
wget https://cloud-images.ubuntu.com/noble/current/noble-server-cloudimg-amd64.img

# Tools for editing the image offline
apt-get install -y libguestfs-tools

# Bake the QEMU guest agent into the image
virt-customize -a noble-server-cloudimg-amd64.img \
    --install qemu-guest-agent \
    --run-command 'systemctl enable qemu-guest-agent'

# Create the VM, import the disk, add cloud-init, set boot and console
qm create 9000 --memory 2048 --core 2 --name ubuntu-2404-cloud \
    --net0 virtio,bridge=vmbr0 --agent enabled=1
qm importdisk 9000 noble-server-cloudimg-amd64.img local-lvm
qm set 9000 --scsihw virtio-scsi-pci --scsi0 local-lvm:vm-9000-disk-0
qm set 9000 --ide2 local-lvm:cloudinit
qm set 9000 --boot c --bootdisk scsi0
qm set 9000 --serial0 socket --vga serial0
qm template 9000
```

What the less obvious steps do:

- `virt-customize` installs `qemu-guest-agent` before the image ever boots. The agent reports the VM's IP back to Proxmox, and Terraform reads the IP from there. Without it, `terraform apply` finishes with an empty IP list.
- `--agent enabled=1` tells Proxmox to query the agent. You need both this and the installed agent.
- `--serial0 socket --vga serial0` is needed because cloud images have no graphical console. Without it the Console tab stays black.

3.5 Terraform files

The repo root has the working versions of all of these.

`provider.tf`:

```
terraform {
  required_providers {
    proxmox = {
      source  = "bpg/proxmox"
      version = ">= 0.60.0"
    }
  }
  required_version = ">= 1.5.0"
}

provider "proxmox" {
  endpoint  = var.proxmox_endpoint
  api_token = var.proxmox_api_token
  insecure  = true   # Proxmox uses a self-signed cert

  ssh {
    agent    = false
    username = "root"
  }
}
```

`variables.tf` sets the endpoint (`https://192.168.1.4:8006/`), node `pve`, `vm_count = 1`, `vm_memory = 12288`, `vm_cores = 6`, `template_id = 9000`, and marks `proxmox_api_token` as `sensitive` so it never prints in plan output.

`main.tf` defines one `proxmox_virtual_environment_vm` resource:

- clones template 9000
- `cpu { type = "host" }` so the guest sees the host CPU's features
- `agent { enabled = true }`
- 40 GB disk on `local-lvm`, NIC on `vmbr0`, DHCP
- cloud-init user `labadmin` with the key from `~/.ssh/id_rsa.pub`
- `lifecycle { ignore_changes = [initialization] }` so later edits don't force a rebuild
- outputs `vm_names` and `vm_ips`

Don't use `admin` as the cloud-init username. Ubuntu 24.04 already has an `admin` group, `useradd` fails, no user gets created, and SSH rejects the key.

`terraform.tfvars` (gitignored, see `terraform.tfvars.example`):

```
proxmox_api_token = "terraform@pve!terraform-token=YOUR-TOKEN-SECRET-HERE"
```

`.gitignore` excludes `*.tfvars`, `terraform.tfstate*` and `.terraform/`. The state file holds secrets in plain text too.

3.6 First run

```
terraform init
terraform plan
terraform apply          # type yes
ssh labadmin@<vm_ip>     # key login, no password
```

Apply takes 20 to 30 seconds.

`start-lab.ps1` and `stop-lab.ps1` in the repo wrap `terraform apply -auto-approve` and `terraform destroy -auto-approve` and print the VM IP. PowerShell blocks local scripts by default, so run this once first:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
.\start-lab.ps1
```

3.7 Resizing the VM

Change `vm_memory`, `vm_cores` or the disk size, then run `terraform plan` and read the summary line before applying:

- `0 to add, 1 to change, 0 to destroy` means an in-place update. Good.
- `1 to add, 0 to change, 1 to destroy` means the VM would be rebuilt. Stop and find out why.

The bpg provider changes memory, CPU and disk in place, though the VM may reboot. cloud-init's growpart grows the root filesystem on boot, so after a disk resize, check `df -h /` once the VM is back up. If it didn't grow, run `growpart` and `resize2fs` by hand.

4. Phase 2: Containerlab and VXLAN-EVPN

Goal: a spine-leaf fabric running VXLAN-EVPN on real Arista EOS, deployed from a YAML file. This is the kind of fabric data centers actually run, and it's not something I'd get to touch in a school lab.

4.1 Resize to 8 GB / 4 cores / 40 GB

Each cEOS container uses about 1.3 GB of RAM, so the 2 GB baseline is too small. See 3.7. After apply:

```
free -h    # about 7.7 Gi
nproc      # 4
df -h /    # about 38 G
```

4.2 Docker

On the lab VM, from Docker's official apt repo:

```
for pkg in docker.io docker-doc docker-compose docker-compose-v2 podman-docker containerd runc; do
    sudo apt-get remove -y $pkg
done

sudo apt-get update
sudo apt-get install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] https://
download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin

sudo docker run hello-world
sudo usermod -aG docker labadmin     # then log out and back in
```

4.3 Containerlab

```
bash -c "$(curl -sL https://get.containerlab.dev)"
containerlab version
```

I used 0.74.

4.4 Arista cEOS-lab image

cEOS-lab is free but needs an Arista account, and the image can't be pulled from a public registry.

1. Register at arista.com. A school email gets approved faster. Gmail sign-ups are sometimes held for manual review. Approval for downloads came in a second email.
2. Software Downloads → cEOS-lab → `cEOS-lab-4.36.0F.tar.xz` (about 500 MB).
3. Copy it to the VM and import it:

```
scp cEOS-lab-4.36.0F.tar.xz labadmin@192.168.1.80:/home/labadmin/
```

```
docker import cEOS-lab-4.36.0F.tar.xz ceos:4.36.0F
docker images | grep ceos
```

It's `docker import` and not `docker pull`, because the file is a filesystem tarball and not a registry image.

4.5 Topology file

`phase2/evpn-fabric.clab.yml`:

```
name: evpn-fabric

mgmt:
  network: clab-mgmt
  bridge: clab-mgmt
  ipv4-subnet: 172.20.20.0/24

topology:
  kinds:
    ceos:
      image: ceos:4.36.0F
    linux:
      image: alpine:3.24

  nodes:
    spine1:
      kind: ceos
      mgmt-ipv4: 172.20.20.11
      startup-config: configs/spine1.cfg
    leaf1:
      kind: ceos
      mgmt-ipv4: 172.20.20.21
      startup-config: configs/leaf1.cfg
    leaf2:
      kind: ceos
      mgmt-ipv4: 172.20.20.22
      startup-config: configs/leaf2.cfg
    host1:
      kind: linux
      mgmt-ipv4: 172.20.20.101
      exec:
        - ip link set eth1 up
        - ip addr add 10.100.100.11/24 dev eth1
    host2:
      kind: linux
      mgmt-ipv4: 172.20.20.102
      exec:
        - ip link set eth1 up
        - ip addr add 10.100.100.12/24 dev eth1

  links:
    - endpoints: ["spine1:eth1", "leaf1:eth1"]
    - endpoints: ["spine1:eth2", "leaf2:eth1"]
    - endpoints: ["leaf1:eth2", "host1:eth1"]
    - endpoints: ["leaf2:eth2", "host2:eth1"]
```

- The `mgmt` block pins the management bridge name. Otherwise it's a new `br-<random>` on every deploy, and Zeek in Phase 4 needs a stable name.
- `startup-config` loads each router's saved config at boot. On the very first build those files don't exist yet, so you deploy without them, configure by hand (4.7), then save them (4.9).
- host1 and host2 are plain Alpine containers that sit on the leaves' access ports. EVPN only learns MACs from real host traffic, so they're required (see 4.12).
- cEOS maps container interface `eth1` to `Ethernet1`, `eth2` to `Ethernet2`, and so on.

4.6 First deploy

```
mkdir -p ~/evpn-lab/configs && cd ~/evpn-lab
# copy in evpn-fabric.clab.yml; remove the startup-config lines for the very first deploy
sudo containerlab deploy -t evpn-fabric.clab.yml
sudo containerlab inspect -t evpn-fabric.clab.yml
```

4.7 Device configs

Design in short:

- Underlay: eBGP over the /31 links, a different ASN per device. Each device advertises its loopback.
- Overlay: a separate EVPN session from each leaf loopback to the spine loopback. All three use `local-as 65100 no-prepend replace-as`, so every EVPN session looks like iBGP in AS 65100, and spine1 reflects routes between the leaves.
- The spine sets `next-hop-self` toward the leaves for IPv4 only. EVPN keeps the original next hop, so the leaves tunnel straight to each other.

The cEOS default login is `admin` / `admin`.

spine1 (`ssh admin@172.20.20.11`, then `enable`, `configure terminal`):

```
interface Loopback0
   ip address 10.0.0.1/32
exit

interface Ethernet1
   no switchport
   ip address 10.1.1.0/31
   description to-leaf1
exit

interface Ethernet2
   no switchport
   ip address 10.1.1.2/31
   description to-leaf2
exit

ip routing

router bgp 65000
   router-id 10.0.0.1
   no bgp default ipv4-unicast
   neighbor LEAF peer group
   neighbor LEAF send-community extended
   neighbor LEAF next-hop-self
   neighbor 10.1.1.1 peer group LEAF
   neighbor 10.1.1.1 remote-as 65001
   neighbor 10.1.1.3 peer group LEAF
   neighbor 10.1.1.3 remote-as 65002
   neighbor EVPN peer group
   neighbor EVPN remote-as 65100
   neighbor EVPN local-as 65100 no-prepend replace-as
   neighbor EVPN update-source Loopback0
   neighbor EVPN ebgp-multihop 3
   neighbor EVPN send-community extended
   neighbor EVPN route-reflector-client
   neighbor 10.0.0.11 peer group EVPN
   neighbor 10.0.0.12 peer group EVPN
   address-family ipv4
      neighbor LEAF activate
      network 10.0.0.1/32
   exit
   address-family evpn
      neighbor EVPN activate
   exit
exit

end
write memory
```

leaf1 (`ssh admin@172.20.20.21`):

```
vlan 100
   name Tenant_A
exit

interface Loopback0
   ip address 10.0.0.11/32
exit

interface Ethernet1
   no switchport
   ip address 10.1.1.1/31
   description to-spine1
exit

interface Ethernet2
   description to-host1
   switchport access vlan 100
exit

interface Vxlan1
   vxlan source-interface Loopback0
   vxlan udp-port 4789
   vxlan vlan 100 vni 10100
exit

ip routing

router bgp 65001
   router-id 10.0.0.11
   no bgp default ipv4-unicast
   neighbor SPINE peer group
   neighbor SPINE send-community extended
   neighbor 10.1.1.0 peer group SPINE
   neighbor 10.1.1.0 remote-as 65000
   neighbor EVPN peer group
   neighbor EVPN remote-as 65100
   neighbor EVPN local-as 65100 no-prepend replace-as
   neighbor EVPN update-source Loopback0
   neighbor EVPN ebgp-multihop 3
   neighbor EVPN send-community extended
   neighbor 10.0.0.1 peer group EVPN
   address-family ipv4
      neighbor SPINE activate
      network 10.0.0.11/32
   exit
   address-family evpn
      neighbor EVPN activate
   exit
   vlan 100
      rd 10.0.0.11:10100
      route-target both 1:10100
      redistribute learned
   exit
exit

end
write memory
```

leaf2 is the same as leaf1 with these values changed:

Setting	leaf2 value
`router bgp`	65002
Loopback0 / router-id	10.0.0.12
Ethernet1	10.1.1.3/31
Spine neighbor	10.1.1.2
Ethernet2 description	to-host2
VLAN 100 RD	10.0.0.12:10100

The full saved configs are in `phase2/configs/`.

4.8 Verify

Wait about 15 seconds for BGP, then:

```
docker exec clab-evpn-fabric-host1 ping -c 5 10.100.100.12
```

The first reply is slower (ARP plus MAC learning). The rest come back in about 2 ms. 0% loss means the overlay works.

On leaf1:

```
show bgp summary
show bgp evpn summary
show bgp evpn route-type mac-ip
show vxlan address-table
```

`show bgp evpn route-type mac-ip` should show two routes: host1's MAC with RD 10.0.0.11:10100 (local) and host2's MAC with RD 10.0.0.12:10100, `Or-ID: 10.0.0.12 C-LST: 10.0.0.1`. The originator is leaf2 and the cluster list shows it came through spine1. `show vxlan address-table` should list host2's MAC as type EVPN on `Vx1` with VTEP `10.0.0.12`.

4.9 Save the configs

```
cd ~/evpn-lab
docker cp clab-evpn-fabric-spine1:/mnt/flash/startup-config configs/spine1.cfg
docker cp clab-evpn-fabric-leaf1:/mnt/flash/startup-config configs/leaf1.cfg
docker cp clab-evpn-fabric-leaf2:/mnt/flash/startup-config configs/leaf2.cfg
wc -l configs/*.cfg
```

Put the `startup-config` lines back in the topology file. From then on every deploy comes up fully configured.

4.10 Start the fabric on boot

The repo has the unit file as `phase2/containerlab-evpn.service`:

```
[Unit]
Description=Containerlab EVPN Fabric Auto-Deploy
After=docker.service network-online.target
Requires=docker.service
Wants=network-online.target

[Service]
Type=oneshot
RemainAfterExit=yes
WorkingDirectory=/home/labadmin/evpn-lab
ExecStartPre=-/usr/bin/containerlab destroy -t /home/labadmin/evpn-lab/evpn-fabric.clab.yml --cleanup
ExecStart=/usr/bin/containerlab deploy -t /home/labadmin/evpn-lab/evpn-fabric.clab.yml
ExecStop=/usr/bin/containerlab destroy -t /home/labadmin/evpn-lab/evpn-fabric.clab.yml --cleanup
TimeoutStartSec=300

[Install]
WantedBy=multi-user.target
```

```
sudo cp containerlab-evpn.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now containerlab-evpn.service
systemctl status containerlab-evpn.service     # active (exited) after 60 to 90 s
```

`ExecStartPre=-` runs a destroy first and ignores the error if there's nothing to destroy, so every boot starts clean. That matters because cEOS doesn't survive a plain `docker start` (see 8).

In Proxmox, VM 100 → Options → Start at boot → Yes. After that, powering on the PC brings up Proxmox, then the VM, then the fabric, in about 2 minutes with no manual steps.

4.11 Commit

Copy the topology, configs and README into `phase2/` in the repo. Before committing, check that `git status` doesn't list the `clab-evpn-fabric/` runtime folder Containerlab creates next to the topology file. It holds certificates, logs and a generated inventory with the default password. The repo `.gitignore` excludes `clab-*/`.

4.12 Phase 2 lessons

Symptom	Cause	Fix
BGP state `Estab(NotNegotiated)`	TCP is up, address family not activated on one side	`neighbor X activate` under the right address family on both ends, check peer group names
EVPN routes created on a leaf but never reach the other leaf	cEOS-lab wouldn't carry EVPN routes across the eBGP overlay sessions	`local-as 65100 no-prepend replace-as` so the overlay is iBGP with spine1 as route reflector
EVPN sessions up, no Type-2 routes	`redistribute learned` exports the bridge MAC table, and SVI-to-SVI pings never filled it	Alpine hosts on access ports
Leaves try to reach each other's MACs through the spine	Next-hop handling is per address family	`next-hop-self` for IPv4 only. EVPN keeps the original next hop

Symptom	Cause	Fix
Fabric gone after a VM reboot	Containerlab containers don't restart on their own	systemd unit (4.10)
CLI and SSH hang after `docker start` on cEOS	EOS's internal Sysdb doesn't recover from a stop and start	`containerlab destroy --cleanup` and deploy again
Bridge name changes every deploy	default `br-<random>` naming	`mgmt: bridge: clab-mgmt`

5. Phase 3: NAPALM and pyATS

Goal: stop checking the fabric by SSHing into each box and reading output. I wanted scripts and tests to do that for me, so I'd know right away if something broke.

5.1 Python environment

```
sudo apt-get update
sudo apt-get install -y python3-pip python3-venv python3-dev build-essential
mkdir -p ~/network-lab-phase3 && cd ~/network-lab-phase3
python3 -m venv venv
source venv/bin/activate
pip install --upgrade pip
pip install napalm "pyats[full]" python-dotenv pytest
```

Ubuntu 24.04 refuses `pip install` outside a venv (PEP 668), so the venv is required. `pyats[full]` takes a few minutes. Check it:

```
python -c "import napalm; print(napalm.__version__)"
python -c "from genie.testbed import load; print('genie ok')"
```

Versions used: Python 3.12.3, napalm 5.1.0, pyats and genie 26.3, pytest 9.0.3.

Each new SSH session needs:

```
cd ~/network-lab-phase3
source venv/bin/activate
set -a; source .env; set +a
```

5.2 Credentials

`.env` (gitignored):

```
NAPALM_USERNAME=admin
NAPALM_PASSWORD=admin
```

The repo has `.env.example` with placeholders. `testbed.yaml` reads the same variables with `%ENV{NAPALM_USERNAME}`.

5.3 01_check_bgp.py

Opens a NAPALM `eos` session (over SSH) to all three routers at once with `ThreadPoolExecutor`, calls `get_bgp_neighbors()`, and prints one table:

```
Device     Neighbor         Remote AS  State  Uptime(s)  Rx Pfx
-------------------------------------------------------------
leaf1      10.0.0.1         65100      Up     ...        ...
leaf1      10.1.1.0         65000      Up     ...        ...
...
```

Expect 8 rows: spine1 has 4 peers (2 underlay, 2 EVPN) and each leaf has 2. A device that fails to connect prints an error row instead of stopping the script.

5.4 02_push_vlan.py

Adds Tenant B to both leaves:

```
vlan 200
   name Tenant_B
interface Vxlan1
   vxlan vlan 200 vni 10200
router bgp <leaf ASN>
   vlan 200
      rd <leaf loopback>:10200
      route-target both 1:10200
      redistribute learned
```

The steps:

1. Open both leaves in parallel and `load_merge_candidate()` with the snippet above.
2. Print `compare_config()` for each leaf.
3. If staging failed anywhere, discard on every leaf and exit.
4. Ask `Commit these changes on both leaves? [y/N]`.
5. `commit_config()` on y, `discard_config()` on anything else.

It's the same plan-then-apply flow as Terraform. Running it a second time shows "no changes, already configured".

5.5 testbed.yaml and test_fabric.py

`testbed.yaml` lists spine1, leaf1 and leaf2 with `os: eos`, SSH to their management IPs, and a `custom.role` of spine or leaf.

`test_fabric.py` has a session-scoped fixture that connects to all three devices once and disconnects at the end. The tests:

Test	Passes when
`test_all_devices_reachable`	all 3 devices connected
`test_underlay_bgp_sessions_established`	

Test	Passes when
	`show ip bgp summary` has at least 2 Estab peers on spine1 and 1 on each leaf
`test_evpn_bgp_sessions_established`	the same counts in `show bgp evpn summary`
`test_vxlan_vni_10100_on_both_leaves`	`show vxlan vni` on both leaves includes 10100
`test_vxlan_vni_10200_on_both_leaves`	both leaves include 10200 (needs 5.4 first)
`test_spine_has_no_vxlan`	spine1 has no VNIs
`test_type2_routes_present_after_ping`	after `docker exec` pings host1 to host2, both leaves have mac-ip routes

```
pytest test_fabric.py -v
```

With the fabric up and the VLAN 200 push committed, the result is 7 passed in about 45 seconds. On a fresh deploy without the push it's 6 passed and 1 failed, which is the correct result.

A session counts only if its state is exactly `Estab`. `Estab(NotNegotiated)` is excluded on purpose, since that's the broken state from 4.12.

The last test makes its own traffic before it checks anything, so it tests MAC learning end to end and doesn't rely on whatever happens to be in the table.

5.6 Phase 3 lessons

Issue	What happened	Workaround
Genie parsers on EOS	`dev.parse(...)` raised `ParserNotFound` or `SchemaEmptyParserError` on this cEOS-lab build	`dev.execute()` for raw text, parse with small regex helpers
`SchemaEmptyParserError`	A parser exists but didn't match the output, usually a CLI format change between versions	Same as above
`show vxlan vni` regex	The first regex missed VNIs whose rows wrap onto continuation lines	Match any line that starts with a number, keep values from 4096 to 16777214
BGP test too loose	Matching "Estab" also matched `Estab(NotNegotiated)`	Exclude NotNegotiated explicitly
`pip install` error	PEP 668 on Ubuntu 24.04	Always use a venv
Copying the project off the VM	`scp -r` pulled the whole venv	Copy files one by one or exclude `venv/`

6. Phase 4: Zeek, ELK and Tailscale

Goal: see the traffic on the lab, and reach the lab from anywhere without opening ports on my home router.

6.1 Resize to 12 GB / 6 cores

Set `vm_memory = 12288` and `vm_cores = 6`, then plan (expect `0 to add, 1 to change, 0 to destroy`) and apply. If the VM reboots, the systemd unit brings the fabric back.

6.2 Zeek

From the openSUSE Build Service repo:

```
sudo apt-get install -y curl gnupg apt-transport-https
echo 'deb http://download.opensuse.org/repositories/security:/zeek/xUbuntu_24.04/ /' | \
    sudo tee /etc/apt/sources.list.d/security:zeek.list
curl -fsSL https://download.opensuse.org/repositories/security:zeek/xUbuntu_24.04/Release.key | \
    gpg --dearmor | sudo tee /etc/apt/trusted.gpg.d/security_zeek.gpg > /dev/null
sudo apt-get update
sudo apt-get install -y zeek

echo 'export PATH=/opt/zeek/bin:$PATH' >> ~/.bashrc
source ~/.bashrc
zeek --version
```

Zeek depends on postfix, which opens a config dialog during install. Choose "No configuration". I don't use email alerts.

Config:

- `/opt/zeek/etc/node.cfg`: change `interface=eth0` to `interface=any`.
- `/opt/zeek/share/zeek/site/local.zeek`: add `@load policy/tuning/json-logs.zeek` at the end.

```
sudo /opt/zeek/bin/zeekctl deploy
sudo /opt/zeek/bin/zeekctl status
```

A warning about the Python `websockets` module is harmless. Zeek 8 has an optional websocket control channel that this setup doesn't use.

Check that logs are coming in:

```
docker exec clab-evpn-fabric-host1 ping -c 10 10.100.100.12
sudo ls /opt/zeek/logs/current/
sudo tail -n 3 /opt/zeek/logs/current/conn.log
```

You should see JSON lines. If `conn.log` never appears, check `interface=any`.

6.3 Elasticsearch and Kibana

`~/elk-stack/docker-compose.yml` (same as `phase4/docker-compose.yml`):

```
services:
  elasticsearch:
    image: docker.elastic.co/elasticsearch/elasticsearch:8.15.0
    container_name: elasticsearch
    environment:
      - discovery.type=single-node
      - xpack.security.enabled=false
      - ES_JAVA_OPTS=-Xms1g -Xmx1g
    ulimits:
      memlock:
        soft: -1
        hard: -1
    volumes:
      - es-data:/usr/share/elasticsearch/data
    ports:
      - "127.0.0.1:9200:9200"
    healthcheck:
      test: ["CMD-SHELL", "curl -fsS http://localhost:9200/_cluster/health || exit 1"]
      interval: 10s
      timeout: 5s
      retries: 10

  kibana:
    image: docker.elastic.co/kibana/kibana:8.15.0
    container_name: kibana
    environment:
      - ELASTICSEARCH_HOSTS=http://elasticsearch:9200
      - SERVER_HOST=0.0.0.0
    ports:
      - "5601:5601"
    depends_on:
      elasticsearch:
        condition: service_healthy

volumes:
  es-data:
```

- Security is off, so Elasticsearch only listens on 127.0.0.1.
- Kibana listens on all interfaces so a laptop on the LAN or tailnet can open it.
- The heap is fixed at 1 GB so Elasticsearch doesn't take most of the VM.
- Kibana waits for the Elasticsearch healthcheck before it starts.

```
cd ~/elk-stack
docker compose up -d
docker compose ps
curl http://localhost:9200/
```

Then open `http://192.168.1.80:5601` from the workstation.

6.4 Filebeat

Filebeat goes on the host, not in Docker, because it reads `/opt/zeek/logs/current/` directly.

```
curl -fsSL https://artifacts.elastic.co/GPG-KEY-elasticsearch | \
    sudo gpg --dearmor -o /etc/apt/trusted.gpg.d/elastic.gpg
echo "deb https://artifacts.elastic.co/packages/8.x/apt stable main" | \
    sudo tee /etc/apt/sources.list.d/elastic-8.x.list
sudo apt-get update
sudo apt-get install -y filebeat

sudo cp /etc/filebeat/filebeat.yml /etc/filebeat/filebeat.yml.original
sudo cp phase4/filebeat.yml /etc/filebeat/filebeat.yml
sudo filebeat test config
```

The config enables the Zeek module for `conn`, `dns`, `notice`, `weird` and `capture_loss` with paths under `/opt/zeek/logs/current/`, sends to `http://localhost:9200`, and sets `index.number_of_replicas: 0`.

```
sudo filebeat setup --modules zeek       # templates, pipelines, stock dashboards (one time, about 30 s)
sudo systemctl enable --now filebeat
curl -s http://localhost:9200/_cat/indices | grep filebeat
```

In Kibana: menu → Analytics → Discover → `filebeat-*` to see the logs live.

6.5 Dashboard

Menu → Analytics → Dashboard → Create dashboard → Create visualization. All four panels are Lens:

Panel	Type	Setup
Connections over time	Line	X `@timestamp`, Y count of records
Top destination IPs	Bar horizontal	Top 10 `destination.ip`, count of records
Protocol split	Pie	slice by `network.transport`
DNS queries over time	Line	X `@timestamp`, Y count, filter `event.dataset : "zeek.dns"`

Set the time range to the last hour and save it as "Network Observability - Phase 4". My screenshot of it is `phase4/phase4-dashboard.png`.

6.6 Tailscale

```
curl -fsSL https://tailscale.com/install.sh | sh
sudo tailscale up            # open the printed URL and sign in
tailscale status
tailscale ip -4              # 100.x.y.z
```

Install Tailscale on the workstation and phone and sign in with the same account. Then:

```
ssh labadmin@<tailscale-ip>
```

On the phone with Wi-Fi off, `http://<tailscale-ip>:5601` loads Kibana over cellular. Everything goes through WireGuard and nothing is forwarded on the home router.

6.7 Phase 4 lessons

Issue	What happened	Fix
Zeek on the mgmt bridge saw almost nothing	Fabric links are veth pairs that never touch the bridge	`interface=any`
No 10.100.100.x flows in Zeek	Overlay traffic is inside VXLAN (UDP 4789) and Zeek logged the outer header	Fine for this lab. Decoding the inner packets needs more Zeek config
Filebeat can't read Zeek logs	`/opt/zeek/logs/current/` is `root:zeek`, `drwxr-sr-x`	Run as root (default) or `usermod -a -G zeek filebeat`
Indices stuck yellow	One node can't hold replicas	`index.number_of_replicas: 0`
Kibana up before Elasticsearch	Startup order	`depends_on` with `condition: service_healthy`
Phone can't see the VM on Tailscale	Signed in with a different identity provider, so it joined a different tailnet	Same login everywhere

7. Operations and recovery

Daily start

```
ssh labadmin@192.168.1.80
systemctl status containerlab-evpn.service
docker exec clab-evpn-fabric-host1 ping -c 3 10.100.100.12
```

Health checks

Layer	Check
Lab VM	`ping 192.168.1.80`
Fabric	`docker exec clab-evpn-fabric-host1 ping -c 3 10.100.100.12`
BGP	`python 01_check_bgp.py`
Everything	`pytest test_fabric.py -v`
Zeek	`sudo /opt/zeek/bin/zeekctl status`
ELK	`docker compose ps` in `~/elk-stack`
Filebeat	`systemctl status filebeat`
Tailscale	`tailscale status`

Rebuild the fabric by hand

```
cd ~/evpn-lab
sudo containerlab destroy -t evpn-fabric.clab.yml --cleanup
sudo containerlab deploy -t evpn-fabric.clab.yml
sleep 30
sudo containerlab inspect -t evpn-fabric.clab.yml
docker exec clab-evpn-fabric-host1 ping -c 5 10.100.100.12
```

Full shutdown

`sudo shutdown -h now` on the lab VM, then shut down the Proxmox node from the web UI. With start-at-boot on, powering the PC back on brings everything back.

8. Troubleshooting

Symptom	Likely cause	Fix
`terraform apply` ends quickly with no IP	Guest agent missing from the template, or `--agent enabled=1` not set	Rebuild the template (3.4)
SSH `Permission denied (publickey)` on a new VM	cloud-init user named `admin`	Use `labadmin` or another name. Read `/var/log/cloud-init.log` for the failing command
Proxmox console tab is black	No serial console on the template	`qm set 9000 --serial0 socket --vga serial0`
`git push` rejects the password	GitHub dropped password auth in 2021	Use a personal access token
Deploy fails with "Subnet 172.20.20.0/24 already in use by Docker network clab"	An older lab without a `mgmt` block left Docker's default `clab` network behind	`docker network rm clab` once no containers use it
cEOS never finishes booting, `Cli` hangs, and agent logs in `/var/log/agents/` show `ModuleNotFoundError`	Damaged cEOS image	`docker rmi ceos:4.36.0F` and import the tarball again
CLI/SSH dead after a container restart	`docker start` on cEOS	Destroy and redeploy
`Estab(NotNegotiated)`	Address family not activated	Add `activate` under the address family
`pytest` shows 6 passed, 1 failed (VNI 10200)	Tenant B push not done since the last deploy	Run `02_push_vlan.py` and commit
No Zeek logs	Wrong capture interface	`interface=any`, then `zeekctl deploy`
Elasticsearch yellow	Replicas on one node	`index.number_of_replicas: 0`

9. Running it on other hardware

Only Phase 1 needs a bare-metal hypervisor. Phases 2 to 4 need a Linux machine with Docker and enough RAM, so they also run on a VM in VMware or VirtualBox, a cloud VM, or WSL2.

Phases	RAM needed	Notes
2 and 3	about 8 GB for the VM	3 cEOS nodes plus 2 Alpine hosts
2, 3 and 4	about 12 GB	adds Elasticsearch, Kibana, Zeek, Filebeat

Other things to know if more than one person runs it:

- Each person needs their own Arista account to download cEOS-lab, because the image can't be shared from a public registry.
- Nokia SR Linux is published at `ghcr.io/nokia/srlinux` with no registration, and Containerlab supports it as the `nokia_srlinux` kind. Swapping vendors means rewriting the device configs and changing the NAPALM driver and pyATS `os` values. The topology, the BGP/EVPN design and the Phase 4 pipeline stay the same.
- Tailscale's free plan is for personal use with a small number of users. A class would use the school's VPN or SSH tunnels instead.
- On a shared network, lock down Kibana (see the security note in `phase4/README.md`).
- All the software here is free: Proxmox (no-subscription repo), Terraform, Containerlab, NAPALM, pyATS, Zeek, and Elasticsearch/Kibana on the free tier.

10. Possible extensions

- CI: a self-hosted GitHub Actions runner on the lab VM that runs the pyATS suite on every push. It has to be self-hosted because GitHub's runners can't reach the lab.
- Streaming telemetry: gNMI from cEOS into Prometheus with Grafana dashboards, next to the Zeek logs.
- Multi-vendor: add an SR Linux or FRR node and run the same NAPALM and pyATS code against it.
- Kubernetes: a small cluster on Proxmox, with the ELK stack moved onto it.
- Zeek VXLAN decoding, so overlay host traffic shows up as host-to-host flows.